APPLIED ARTIFICIAL INTELLIGENCE LABORATORY (ACAD04)

WEEK - 5 CODES

5.1 Handling Time-Series Data with Pandas

Program :

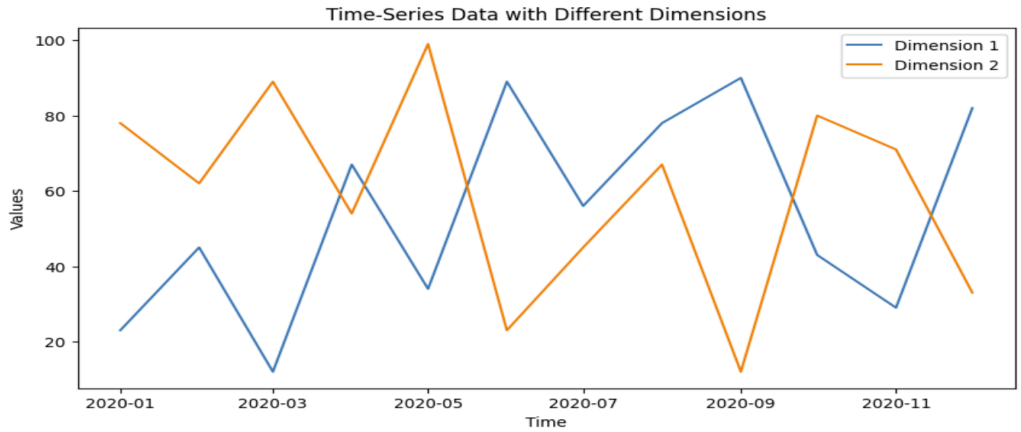
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def read_data(file_path):
    """Reads the data from the input CSV file."""
    df = pd.read_csv(file_path)
    return df

def convert_to_date(date_str):
    """Lambda function to convert strings to Pandas datetime format."""
    return pd.to_datetime(date_str)

def generate_time_index(start_date, periods, freq='M'):
    """Creates a list of indices with dates using the start date and a monthly
frequency."""
    return pd.date_range(start=start_date, periods=periods, freq=freq)

def create_time_series(df, date_column, data_columns):
    """Creates pandas time-series data using timestamps."""
    df[date_column] = df[date_column].apply(convert_to_date)
    df.set_index(date_column, inplace=True)
    return df[data_columns]

def plot_time_series(data, title):
    """Plots the time-series data."""
    plt.figure(figsize=(10, 5))
    for column in data.columns:
        plt.plot(data.index, data[column], label=column)
    plt.xlabel("Time")
    plt.ylabel("Values")
```

```
    plt.title(title)
    plt.legend()
    plt.show()

def main():
    file_path = "timeseries_data.csv"  # Specify the input file
    date_column = "Date"  # Column containing timestamps
    data_columns = ["Dimension 1", "Dimension 2"]  # Specify the data columns

    df = read_data(file_path)
    time_series_data = create_time_series(df, date_column, data_columns)
    plot_time_series(time_series_data, "Time-Series Data with Different Dimensions")

if __name__ == "__main__":
    main()
```

Sample DataSet :

Date,Dimension 1,Dimension 2
2020-01-01,23,78
2020-02-01,45,62
2020-03-01,12,89
2020-04-01,67,54
2020-05-01,34,99
2020-06-01,89,23
2020-07-01,56,45
2020-08-01,78,67
2020-09-01,90,12
2020-10-01,43,80
2020-11-01,29,71
2020-12-01,82,33

Output :



5.2 Slicing Time-Series Data

Program :

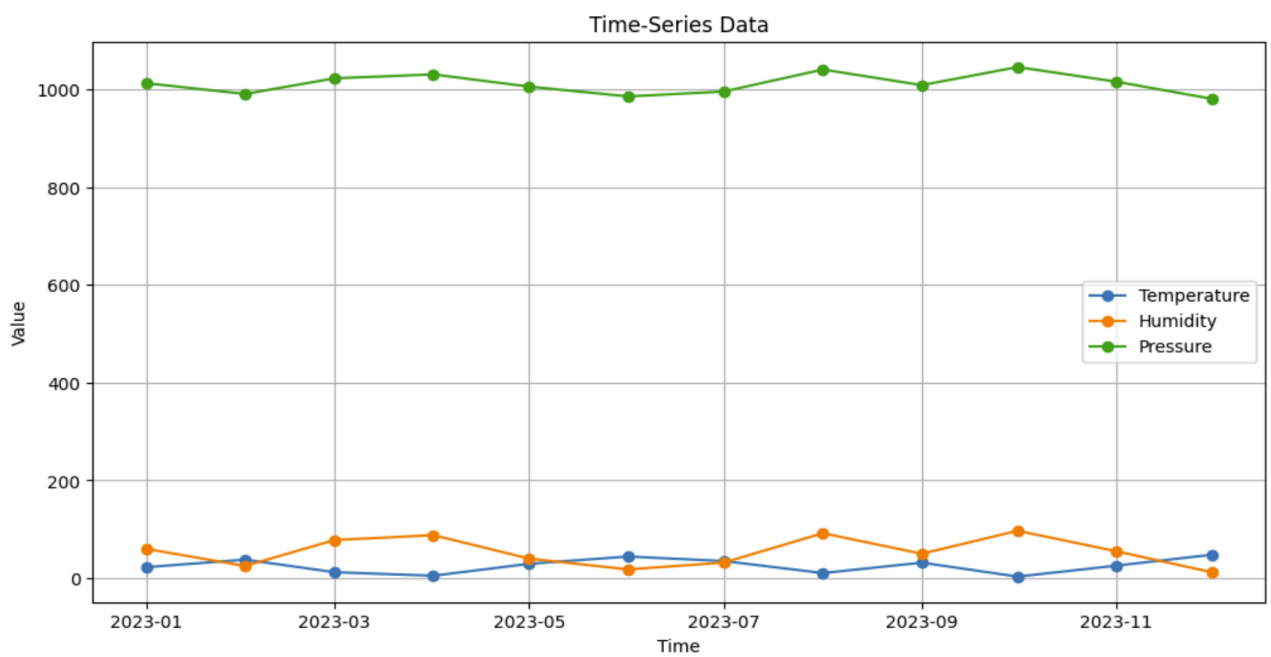
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def read_data(file_path):
    return pd.read_csv(file_path)

# Lambda function to convert strings to Pandas datetime format
convert_to_date = lambda date_str: pd.to_datetime(date_str)

def process_time_series(file_path):
    df = read_data(file_path)

    # Get start date from the first line of the input file
    start_date = convert_to_date(df.iloc[0, 0])

    # Create a list of indices with a monthly frequency
    end_date = convert_to_date(df.iloc[-1, 0])
    date_range = pd.date_range(start=start_date, end=end_date, freq='M')

    # Create pandas data series using the timestamps
    df['Date'] = pd.to_datetime(df['Date'])
    df.set_index('Date', inplace=True)

    # Plot time-series data
    plt.figure(figsize=(12, 6))
    for column in df.columns:
        plt.plot(df.index, df[column], marker='o', linestyle='-', label=column)

    plt.title('Time-Series Data')
    plt.xlabel('Time')
    plt.ylabel('Value')
    plt.legend()
    plt.grid()
    plt.show()

if __name__ == "__main__":
    file_path = 'your_timeseries_data.csv'  # Specify input file
```

```
    process_time_series(file_path)
```

Sample DataSet :

Date,Temperature,Humidity,Pressure
2023-01-01,22.5,60,1012
2023-02-01,38.0,25,990
2023-03-01,12.2,78,1022
2023-04-01,5.1,88,1030
2023-05-01,29.5,40,1005
2023-06-01,44.3,18,985
2023-07-01,35.1,32,995
2023-08-01,10.5,92,1040
2023-09-01,31.8,50,1008
2023-10-01,3.3,97,1045
2023-11-01,25.7,55,1015
2023-12-01,48.1,12,980

Output :



5.3 Operating on Time-Series Data

Program :

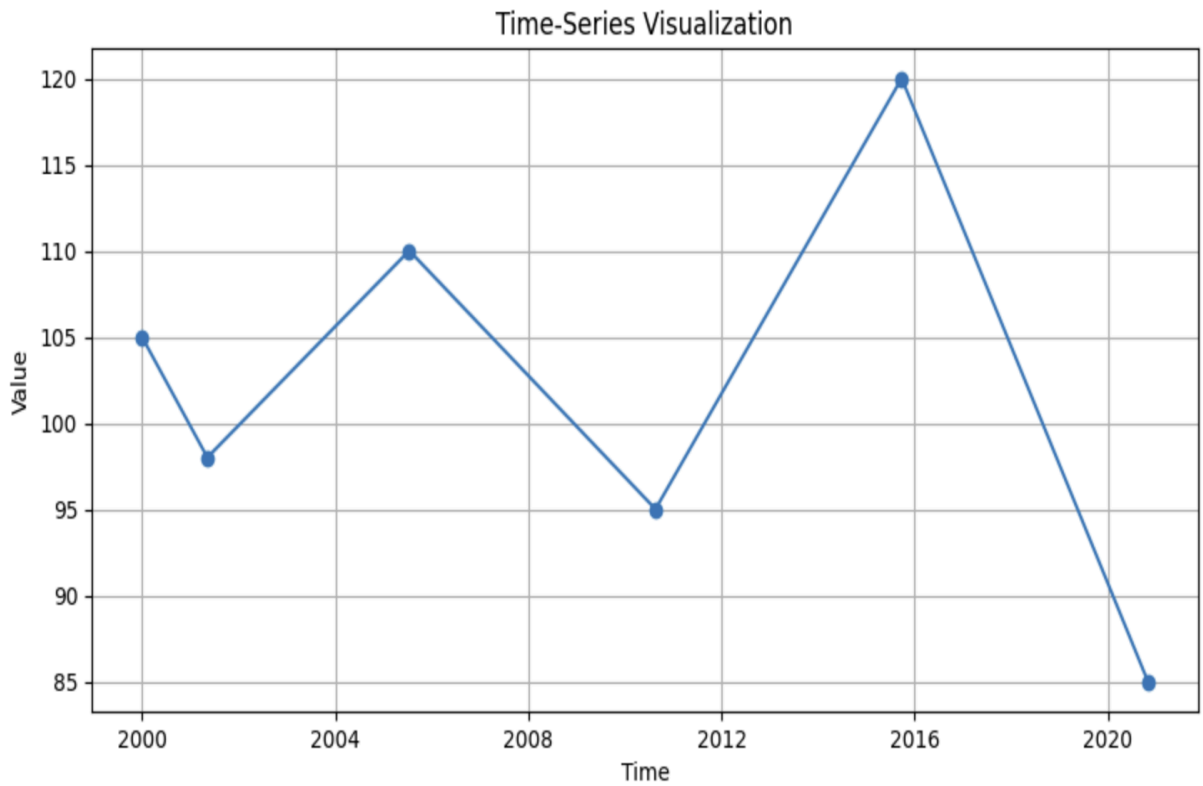
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def read_data(filename):
    """Reads a CSV file and returns a pandas DataFrame."""
    return pd.read_csv(filename)

def process_timeseries(filename, col1_index, col2_index, start_year=None,
end_year=None, filter_condition=None):
    """
    Loads a time-series dataset, selects specific columns by index, filters data, and
visualizes it.
    """
    try:
        # Load data
        df = read_data(filename)

        # Validate column indexes
        if len(df.columns) < max(col1_index, col2_index) + 1:
            raise ValueError("The CSV file does not contain the required columns.")

        # Selecting required columns
        df = df.iloc[:, [col1_index, col2_index]]
        df.columns = ['Time', 'Value']

        # Convert Time column to datetime
        df['Time'] = pd.to_datetime(df['Time'], errors='coerce')
        df.dropna(subset=['Time'], inplace=True)

        # Remove missing values
        df.dropna(inplace=True)

        # Apply year filtering if specified
        if start_year:
            df = df[df['Time'].dt.year >= start_year]
        if end_year:
            df = df[df['Time'].dt.year <= end_year]
```

```python
        # Apply additional filter conditions if specified
        if filter_condition:
            df = df[filter_condition(df)]

        # Plot the data
        plt.figure(figsize=(10, 5))
        plt.plot(df['Time'], df['Value'], marker='o', linestyle='-')
        plt.xlabel("Time")
        plt.ylabel("Value")
        plt.title("Time-Series Visualization")
        plt.grid()
        plt.savefig("timeseries_plot.png")  # Save the plot
        plt.show()

        # Display summarized results
        print("Summary Statistics:")
        print(df.describe())

    except Exception as e:
        print(f"Error: {e}")

# Example usage
if __name__ == "__main__":
    process_timeseries("data.csv", 2, 3, start_year=2000, end_year=2020)
```

Sample DataSet :

ID,Category,Date,Value
1,A,1999-12-31,100
2,A,2000-01-01,105
3,B,2001-05-10,98
4,B,2005-07-15,110
5,C,2010-08-20,95
6,C,2015-09-25,120
7,A,2020-10-30,85
8,A,2021-11-10,130

Output :




```
Summary Statistics:
                      Time       Value
count                    6    6.000000
mean   2009-01-05 12:00:00  102.166667
min    2000-01-01 00:00:00   85.000000
25%    2002-05-26 18:00:00   95.750000
50%    2008-02-01 00:00:00  101.500000
75%    2014-06-16 12:00:00  108.750000
max    2020-10-30 00:00:00  120.000000
std                    NaN   12.254251
```

5.4 Extracting Statistics from Time-Series Data

Program :

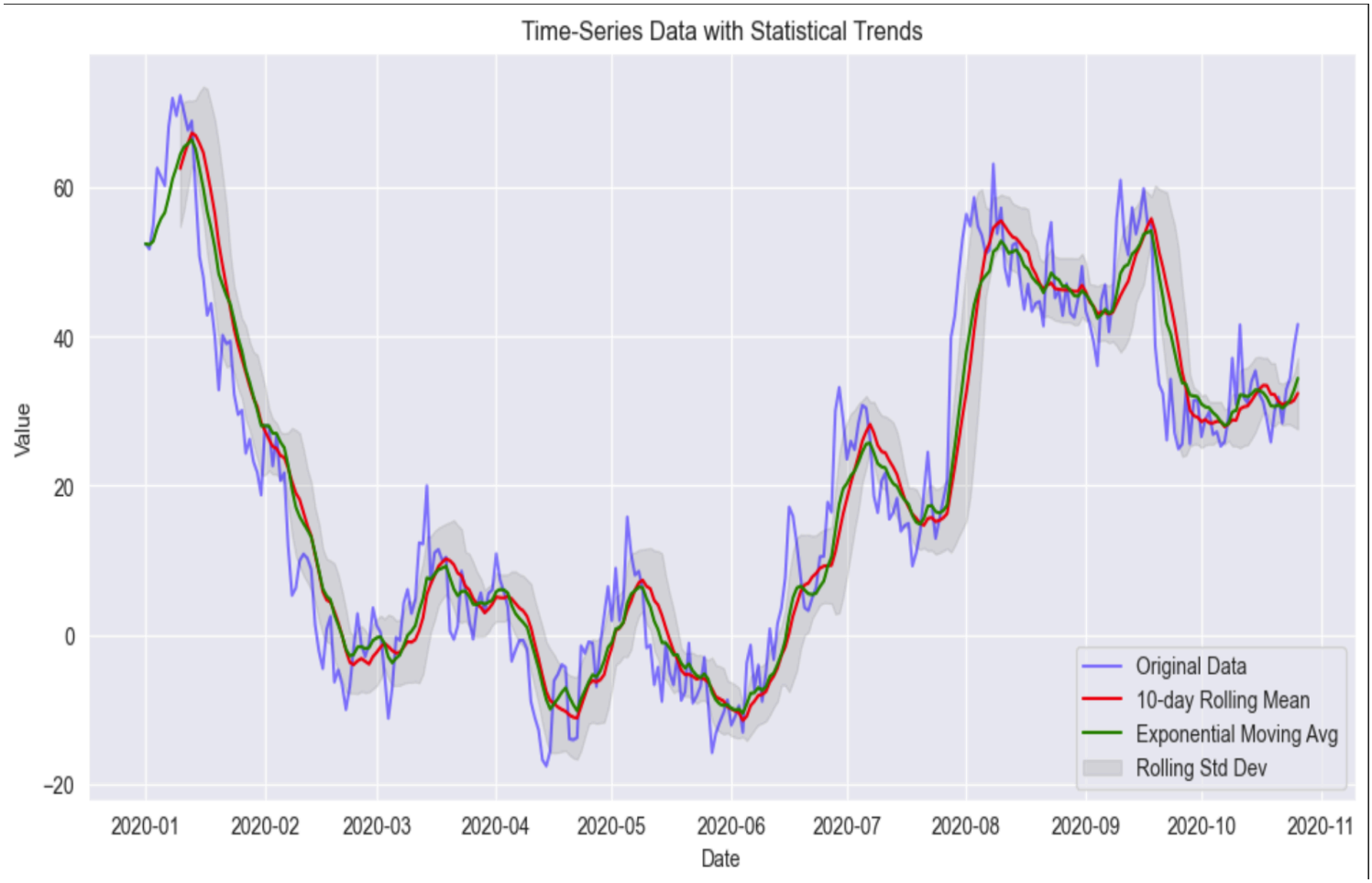
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Generate synthetic time-series data
def generate_time_series_data(n=300):
    np.random.seed(42)
    time = pd.date_range(start='1/1/2020', periods=n, freq='D')
    values = np.cumsum(np.random.randn(n) * 5) + 50  # Random walk with trend
    return pd.DataFrame({'Date': time, 'Value': values})

def calculate_statistics(df):
    df['Rolling_Mean'] = df['Value'].rolling(window=10).mean()
    df['Rolling_Std'] = df['Value'].rolling(window=10).std()
    df['Exp_Moving_Avg'] = df['Value'].ewm(span=10, adjust=False).mean()
    return df

def compute_additional_statistics(df):
    print("Maximum Values for Each Dimension:\n", df[['Value', 'Rolling_Mean',
'Rolling_Std', 'Exp_Moving_Avg']].max(), "\n")
    print("Minimum Values for Each Dimension:\n", df[['Value', 'Rolling_Mean',
'Rolling_Std', 'Exp_Moving_Avg']].min(), "\n")
    print("Overall Mean:", df[['Value', 'Rolling_Mean', 'Rolling_Std',
'Exp_Moving_Avg']].mean().mean(), "\n")

    df['Row_Wise_Mean'] = df[['Value', 'Rolling_Mean', 'Exp_Moving_Avg']].mean(axis=1)
    print("Row-wise Mean (First 10 Rows):\n", df[['Date', 'Row_Wise_Mean']].head(10),
"\n")

    print("Correlation Coefficients:\n", df[['Value', 'Rolling_Mean', 'Rolling_Std',
'Exp_Moving_Avg']].corr(), "\n")

def visualize_statistics(df):
    plt.figure(figsize=(12, 6))
    sns.set_style('darkgrid')
```

```
    # Plot original data
    plt.plot(df['Date'], df['Value'], label='Original Data', color='blue', alpha=0.5)

    # Overlay Rolling Mean
    plt.plot(df['Date'], df['Rolling_Mean'], label='10-day Rolling Mean', color='red')

    # Overlay Exponential Moving Average
    plt.plot(df['Date'], df['Exp_Moving_Avg'], label='Exponential Moving Avg',
color='green')

    plt.fill_between(df['Date'], df['Rolling_Mean'] - df['Rolling_Std'],
df['Rolling_Mean'] + df['Rolling_Std'], color='gray', alpha=0.2, label='Rolling Std Dev')

    plt.title('Time-Series Data with Statistical Trends')
    plt.xlabel('Date')
    plt.ylabel('Value')
    plt.legend()
    plt.show()

if __name__ == "__main__":
    df = generate_time_series_data()
    df = calculate_statistics(df)
    compute_additional_statistics(df)
    visualize_statistics(df)
```

Output :

```
Maximum Values for Each Dimension:
 Value              72.403056
Rolling_Mean       67.356711
Rolling_Std        17.628733
Exp_Moving_Avg     66.470853
dtype: float64

Minimum Values for Each Dimension:
 Value             -17.634727
Rolling_Mean      -11.510514
Rolling_Std         2.177035
Exp_Moving_Avg    -10.556821
dtype: float64
```

```
Overall Mean: 16.362445545155733

Row-wise Mean (First 10 Rows):
       Date      Row_Wise_Mean
0 2020-01-01      52.483571
1 2020-01-02      52.075063
2 2020-01-03      53.937267
3 2020-01-04      58.635933
4 2020-01-05      58.673190
5 2020-01-06      58.490855
6 2020-01-07      63.486445
...
Rolling_Mean    0.938626      1.000000     0.239963        0.996655
Rolling_Std     0.264809      0.239963     1.000000        0.241877
Exp_Moving_Avg  0.959415      0.996655     0.241877        1.000000
```

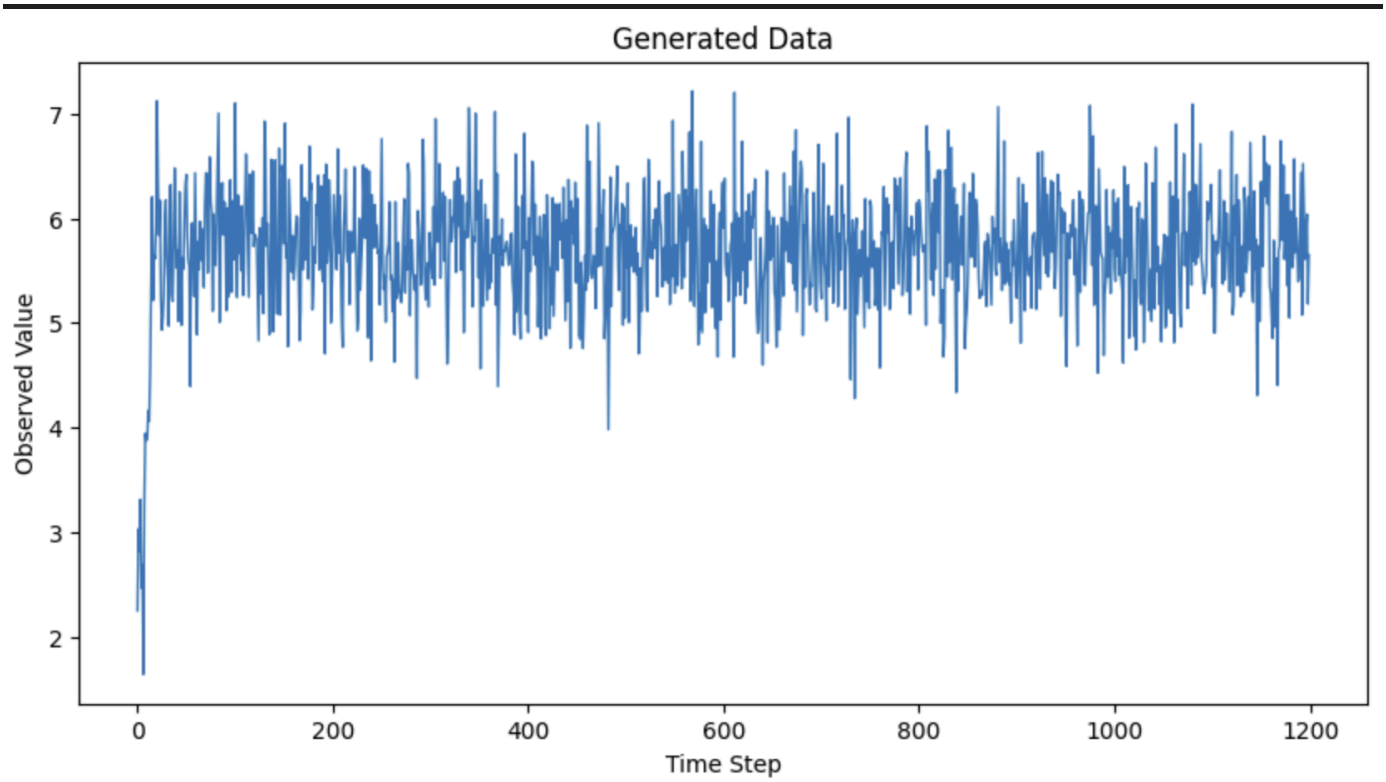


5.5 Generating Data using Hidden Markov Models

Program :

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from hmmlearn import hmm  # Import HMM library

# Function to read data from CSV file
def read_data(filename):
    data = pd.read_csv(filename)
    return data

# Define input filename
filename = "data_1.csv"  # Replace with actual file path

data = read_data(filename)

# Extract the third column for training (assuming zero-based index)
observations = data.iloc[:, 2].values.reshape(-1, 1)

# Create a Gaussian HMM with 5 components and diagonal covariance
model = hmm.GaussianHMM(n_components=5, covariance_type="diag", n_iter=100,
random_state=42)

# Train the HMM
model.fit(observations)

# Print mean and variance values for each component of the HMM
print("Training the Hidden Markov Model...")
print("\nMeans and variances:")
for i in range(model.n_components):
    print(f"Hidden state {i + 1}")
    print(f"Mean = {model.means_[i][0]}")
    print(f"Variance = {model.covars_[i][0]}")

# Generate and visualize samples
samples, _ = model.sample(1200)
plt.figure(figsize=(10, 5))
plt.plot(samples, lw=1)
plt.title("Generated Data")
```

```
plt.xlabel("Time Step")
plt.ylabel("Observed Value")
plt.show()
```

Output :

```
Fitting a model with 34 free scalar parameters with only 15 data points will result
in a degenerate solution.
Training the Hidden Markov Model...

Means and variances:
Hidden state 1
Mean = 4.59962140610021
Variance = [0.01012696]
Hidden state 2
Mean = 2.9002068697659533
Variance = [0.1651968]
Hidden state 3
Mean = 2.700206869754801
Variance = [0.1651968]
Hidden state 4
Mean = 5.728362649810654
Variance = [0.2522324]
Hidden state 5
Mean = 4.033257089572678
Variance = [0.04555387]
```